Lab 8: Binary Trees and Binary Search Trees

Learning Objectives

- Complete recursive binary tree traversals and implement BST insertion and search.

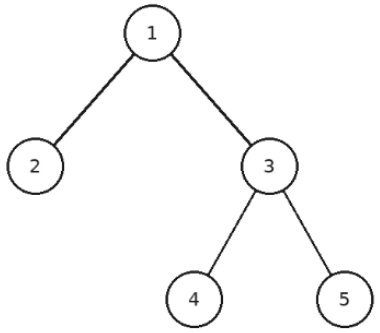
Starter Files

Download both starter files from Canvas for the Activity 1 : ***Lab8_Activity1.cpp*** and Activity 2: ***Lab8_Activity2.cpp.***

Activity 1: Binary Tree Traversals

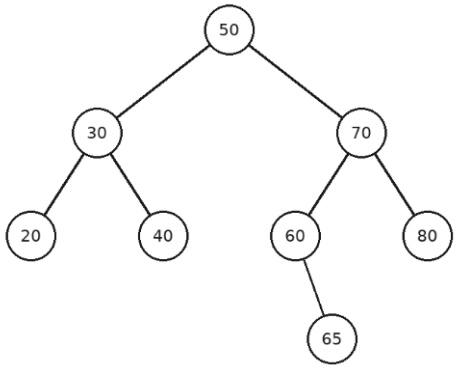
The tree builder uses this preorder list: {1, 2, -1, -1, 3, 4, -1, -1, 5, -1, -1}. A value of -1 marks a missing child. Inorder is demonstrated in class. Complete preorder and postorder; stop at nullptr and print each node followed by a space.

Traversal	Expected output
Preorder	1 2 3 4 5
Inorder	2 1 4 3 5
Postorder	2 4 5 3 1



Activity 2: BST Insertion and Search

Insert in order: 50, 30, 70, 20, 40, 60, 80, 65. At each node, go left if the value is smaller; otherwise go right. Use the slide signatures Node* insert(Node*, int) and Node* search(Node*, int). Search returns the matching node or nullptr.




```
Node* bstRoot = nullptr;              // Start with an empty BST.
bstRoot = insert(bstRoot, value);     // Save the returned root.
Node* result = search(bstRoot, key);  // nullptr means not found.
// In insert(), allocate a node with new Node(value).
```

<u>Expected Output:</u>

- Inorder print: Expected output=20 30 40 50 60 65 70 80 (Inorder visits **left subtree → node → right subtree**)
- Search for 40: Found. Search for 55: Not found